AGM Salary Calculator · Complete Technical Skill File

Version 2.0 | Last Updated: April 12, 2026

Prepared for: Developers, DevOps, Stakeholders

TABLE OF CONTENTS

1 Project Overview
2 Architecture Diagram
3 Directory Structure
4 Tech Stack & Dependencies
5 Environment Variables
6 Database Schema (MongoDB)
7 Authentication Flow
8 API Reference
9 Frontend Pages & Components
10 Salary Calculation Engine
11 Advance Management Module
12 Security Measures
13 Deployment Guide
14 Testing
15 Known Constraints & Decisions

---

1. PROJECT OVERVIEW

AGM Salary Calculator is a web application for managing monthly employee salary processing.

Core Modules: | Module | Purpose | |--------|---------| | Employee Master | CRUD for employee records (code, name, department, salary) | | Attendance Upload | Parse Excel attendance sheets (IN/OUT times per day) | | Salary Configuration | Configurable rules (OT, sandwich, half-day, short hours) | | Salary Report | Calculated salary table, PDF/Excel export, salary slips | | Salary History | Save, compare, track growth across months (MongoDB) | | Advance Management | Upload CSV/Excel advance data, auto-deduct from salary | | Authentication | JWT-based login, protected routes, rate-limited |

What was REMOVED (do NOT restore):

- Invoice Extractor (PDF parsing module)
- OrderHub (e-commerce order consolidation)

---

2. ARCHITECTURE DIAGRAM

```
+-----------------------------------------------+
|                    BROWSER                     |
|                                                |
|   React 19 + Tailwind + Shadcn/UI             |
|   Pages: Login, Dashboard, Employees,         |
|          Attendance, Config, Reports, Advance  |
|                                                |
```

```
|   AuthContext (JWT in sessionStorage)             |
|   AppContext  (employees, attendance, results)    |
|                                                   |
+---------+--------+-------------------------------+
          |        |
          | HTTPS  | Authorization: Bearer <JWT>
          |        |
+---------v--------v-------------------------------+
|                                                   |
|   FastAPI (Python 3.11) · Port 8001              |
|                                                   |
|   Middleware Stack (order matters):               |
|   1. CORS (restricted origins)                   |
|   2. Security Headers (CSP, X-Frame, etc.)       |
|   3. Rate Limiting (slowapi)                     |
|                                                   |
|   Routers:                                        |
|   /api/auth/*     · auth.py (login, verify)      |
|   /api/employees  · server.py (CRUD)             |
|   /api/salary/*   · server.py (history, compare) |
|   /api/advance/*  · advance_api.py (upload, list)|
|                                                   |
+---------+---------------------------------------+
          |
          | motor (async)
          |
+---------v---------------------------------------+
|                                                   |
|   MongoDB                                         |
|   Database: test_database (from DB_NAME env)     |
|                                                   |
|   Collections:                                    |
|   - employees                                     |
|   - salary_records                                |
|   - salary_advances                               |
|   - advance_uploads                               |
|                                                   |
+---------------------------------------------------+
```

---

3. DIRECTORY STRUCTURE

```
/app/
··· backend/
·   ··· server.py            # Main FastAPI app (employees, salary
CRUD, middleware)
·   ··· auth.py              # JWT authentication (login, verify, token
logic)
·   ··· advance_api.py       # Advance CSV/Excel upload + listing
·   ··· rate_limit.py        # Shared slowapi Limiter instance
·   ··· requirements.txt     # Python dependencies (pip freeze)
·   ··· .env                 # Environment variables (DO NOT commit)
·   ··· tests/
```

```
·       ··· test_auth_security.py
·       ··· test_rate_limit_localstorage.py
·       ··· test_salary_history.py
·
··· frontend/
    ··· public/
    ·   ··· index.html       # PostHog config (disabled), Emergent
scripts
    ··· package.json
    ··· src/
        ··· App.js           # Routes: PrivateRoute wrappers,
AuthProvider
        ··· App.css
        ··· index.js         # React entry point
        ··· index.css        # Tailwind imports
        ·
        ··· context/
        ·   ··· AuthContext.jsx   # JWT login/logout, token in
sessionStorage
        ·   ··· AppContext.js     # Global state: employees, config,
attendance, results
        ·
        ··· components/
        ·   ··· Layout.jsx        # Sidebar navigation + logout button
        ·   ··· PrivateRoute.jsx  # Redirect to /login if
unauthenticated
        ·   ··· ui/               # Shadcn/UI components (button, card,
table, etc.)
        ·
        ··· pages/
        ·   ··· Login.jsx              # Username/password form
        ·   ··· Dashboard.jsx          # Overview stats, reset salary
data
        ·   ··· EmployeeMaster.jsx     # Employee CRUD table
        ·   ··· AttendanceUpload.jsx   # Excel drag-drop, month/year
selector
        ·   ··· SalaryConfiguration.jsx # Config toggles + Calculate
button
        ·   ··· SalaryReport.jsx       # Results table, export,
history, compare
        ·   ··· AdvanceManagement.jsx  # CSV upload, advance records
table
        ·
        ··· utils/
            ··· constants.js        # DEFAULT_CONFIG, STORAGE_KEYS,
TRANSLATIONS
            ··· storage.js          # Server-only sync (no
localStorage for sensitive data)
            ··· salaryCalculator.js # Core salary math engine
            ··· excelParser.js      # Attendance Excel parser (IN/OUT
extraction)
            ··· exportUtils.js      # Excel, PDF, salary slip
generators
```

---

4. TECH STACK & DEPENDENCIES

Backend (Python 3.11)

Package	Version	Purpose
fastapi	0.110.1	Web framework
uvicorn	0.25.0	ASGI server
motor	3.3.1	Async MongoDB driver
python-jose	3.5.0	JWT encode/decode
slowapi	0.1.9	Rate limiting
pandas	3.0.0	CSV/Excel parsing (advance uploads)
openpyxl	3.1.5	Excel file support
pydantic	2.12.5	Data validation + serialization
python-dotenv	1.2.1	.env file loading
python-multipart	0.0.22	File upload support

Frontend (Node.js)

Package	Version	Purpose
react	^19.0.0	UI framework
react-dom	^19.0.0	DOM rendering
react-router-dom	^7.5.1	Client-side routing
tailwindcss	^3.4.17	CSS utility framework
lucide-react	^0.507.0	Icons
sonner	^2.0.3	Toast notifications
xlsx	^0.18.5	Excel file parsing (attendance)
jspdf	^4.1.0	PDF generation
Shadcn/UI	·	Pre-built UI components (in /components/ui/)

---

5. ENVIRONMENT VARIABLES

Backend (`/app/backend/.env`)

```
MONGO_URL="mongodb://localhost:27017"      # MongoDB connection
string
DB_NAME="test_database"                     # Database name
APP_USERNAME="admin"                        # Login username
APP_PASSWORD="admin123"                     # Login password
JWT_SECRET_KEY="your-secret-key-here"       # JWT signing secret
(change in production!)
```

Frontend (`/app/frontend/.env`)

```
REACT_APP_BACKEND_URL=https://your-domain.com   # Backend API base
URL
```

Rules:

- NEVER hardcode credentials in source code
- NEVER commit `.env` files to git
- Change `JWT_SECRET_KEY` and `APP_PASSWORD` for production
- Frontend reads `REACT_APP_BACKEND_URL` at build time

---

6. DATABASE SCHEMA (MongoDB)

Collection: `employees`

```
{
  "code": "101",                    // Unique employee code
```

```
(string)
  "name": "Ramesh Kumar",          // Full name
  "department": "Production",      // Optional department
  "salary": 15000,                 // Monthly salary (number)
  "dateOfJoining": "2024-01-15",   // Optional (string)
  "status": "active",              // "active" | "inactive"
  "onlySundayNoOT": false          // If true: no OT calculated,
only Sunday pay
}
```

Index: `code` (unique)

Collection: `salary_records`

```
{
  "record_id": "2026-02",          // "{year}-{month}" (unique,
used for upsert)
  "month": 2,
  "year": 2026,
  "daysInMonth": 28,
  "employeeCount": 45,
  "totalPayout": 450000,
  "config": { ... },               // Salary config snapshot used
for this calculation
  "savedAt": "2026-02-28T10:00:00Z",
  "updatedAt": "2026-02-28T12:00:00Z",
  "employees": [
    {
      "code": "101",
      "name": "Ramesh Kumar",
      "department": "Production",
      "baseSalary": 15000,
      "presentDays": 24,
      "absentDays": 2,
      "sandwichDays": 0,
      "sundayWorking": 2,
      "otHours": 5.5,
      "shortHours": 1.0,
      "netOTHours": 4.5,
      "totalPayableDays": 26.5,
      "totalSalary": 14196,
      "perDaySalary": 535.71,
      "otAmount": 267,
      "deductions": 0
    }
  ]
}
```

Index: `record_id` (unique, upsert key)

Collection: `salary_advances`

```
{
  "date": "15/03/2026",            // Transaction date (string,
```

```
multiple formats supported)
  "name": "Ramesh Kumar",          // Matched employee name
  "employeeCode": "101",           // Matched employee code
  "amount": 5000,                  // Advance amount (number)
  "type": "Salary",                // Always "Salary" (others are
filtered out)
  "uid": "ADV001",                 // Optional unique ID (used for
upsert)
  "syncStatus": "Done",
  "uploadedAt": "2026-03-15T10:00:00Z"
}
```

Upsert key: `uid` (if provided) or duplicate check on `date + employeeCode + amount`

Collection: `advance_uploads`

```
{
  "filename": "advance_march.csv",
  "timestamp": "2026-03-15T10:00:00Z",
  "total_rows": 50,
  "matched": 40,
  "updated": 5,
  "skipped": 3,
  "errors": 2
}
```

---

7. AUTHENTICATION FLOW

```
...............          ...................          .............
·   Browser    ·          ·  FastAPI Backend ·          ·  MongoDB  ·
·  (React)     ·          ·                  ·          ·           ·
...............          ...................          .............
       ·                          ·                          ·
       ·  1. POST /api/auth/login  ·                          ·
       ·  {username, password}     ·                          ·
       ........................> ·                          ·
       ·                          ·                          ·
       ·           Rate limit check (5/min per IP)           ·
       ·           Validate vs APP_USERNAME / APP_PASSWORD    ·
       ·                          ·                          ·
       ·  2. {access_token, type,  ·                          ·
       ·      expires_in}          ·                          ·
       · <........................                           ·
       ·                          ·                          ·
       ·  3. Store token in        ·                          ·
       ·     sessionStorage        ·                          ·
       ·     ('auth_token')        ·                          ·
       ·                          ·                          ·
       ·  4. GET /api/employees    ·                          ·
       ·  Authorization: Bearer xx ·                          ·
       ........................> ·                          ·
       ·                          ·  5. Depends(get_current ·
```

```
·                            ·     _user) validates JWT·
·                            ·                          ·
·                            ·  6. Query employees     ·
·                            ·........................>·
·                            ·                          ·
·  7. {success, data: [...]}·  <......................·
· <........................·                          ·
·                            ·                          ·
·  On 401: clear token,     ·                          ·
·  redirect to /login       ·                          ·
```

JWT Token Structure:

```
{
  "sub": "admin",           // Username
  "exp": 1744416000,        // Expiry (8 hours from creation)
  "iat": 1744387200         // Issued at
}
```

Token Storage: `sessionStorage` (cleared on tab close, not persisted across sessions)

---

8. API REFERENCE

All endpoints (except `/api/auth/login`, `/health`) require:

```
Authorization: Bearer <jwt_token>
```

8.1 Authentication

POST `/api/auth/login`

Rate limit: **5 requests/minute** per IP

```
Request:  {"username": "admin", "password": "admin123"}
Response: {"access_token": "eyJ...", "token_type": "bearer",
"expires_in": 28800}
Error:    {"detail": "Invalid username or password"} (401)
```

GET `/api/auth/verify`

```
Response: {"valid": true, "username": "admin"}
Error:    {"detail": "Not authenticated"} (401)
```

POST `/api/auth/logout`

```
Response: {"success": true, "message": "Logged out successfully"}
```

8.2 Employees

GET `/api/employees`

```
Response: {"success": true, "message": "Employees loaded", "data":
```

```
[{...}]}
```

POST `/api/employees`

Rate limit: **10/minute** Replaces ALL employees (bulk save).

```
Request:

[{"code":"101","name":"Ramesh","salary":15000,"status":"active","onlySundayNoOT":fals
Response: {"success": true, "message": "Saved 45 employees"}
```

POST `/api/employees/add`

```
Request:  {"code":"102","name":"Suresh","salary":12000}
Response: {"success": true, "message": "Employee added"}
```

PUT `/api/employees/{code}`

```
Request:  {"code":"101","name":"Ramesh Kumar","salary":16000,...}
Response: {"success": true, "message": "Employee updated"}
```

DELETE `/api/employees/{code}`

```
Response: {"success": true, "message": "Employee deleted"}
```

8.3 Salary

POST `/api/salary/save`

Rate limit: **10/minute**. Upserts by `{year}-{month}`.

```
Request: {
  "month": 2, "year": 2026, "daysInMonth": 28,
  "employees": [{...}], "totalPayout": 450000, "config": {...}
}
Response: {"success": true, "message": "Salary saved for 2/2026",
"record_id": "2026-02"}
```

GET `/api/salary/history`

```
Response: {"success": true, "data": [


{"record_id":"2026-02","month":2,"year":2026,"totalPayout":450000,"employeeCount":45,
]}
```

GET `/api/salary/history/{year}/{month}`

```
Response: {"success": true, "data": {full record with employees
array}}
```

PUT `/api/salary/history/{year}/{month}/{emp_code}`

Updates one employee's salary in a saved record.

```
Request:  {"presentDays": 25, "totalSalary": 15000}
Response: {"success": true, "message": "Updated salary for 101"}
```

DELETE `/api/salary/history/{year}/{month}`

```
Response: {"success": true, "message": "Deleted salary record for
2/2026"}
```

GET `/api/salary/compare/{year1}/{month1}/{year2}/{month2}`

```
Response: {"success": true, "data": {
  "summary": {"totalPayout1": 400000, "totalPayout2": 450000,
"difference": 50000},
  "employees":
[{"code":"101","salary1":14000,"salary2":15000,"difference":1000}]
}}
```

GET `/api/salary/employee/{emp_code}/growth`

```
Response: {"success": true, "data": {
  "employeeCode": "101",
  "history": [{"month":1,"year":2026,"totalSalary":14000}, ...],
  "totalGrowth": 2000, "avgMonthlyGrowth": 500, "monthsTracked": 4
}}
```

8.4 Advance Management

POST `/api/advance/upload`

Multipart form: `file` (CSV/Excel) + optional `employees_json` (JSON string).

```
Response: {
  "success": true, "message": "Processed 50 rows",
  "matched": 40, "updated": 5, "skipped": 3, "errors": 2,
  "details": [{"row": 2, "status": "inserted"}, ...]
}
```

GET `/api/advance/list?month=3&year=2026`

```
Response: {
  "advances": [{...}],
  "stats": {"total": 7, "totalAmount": 18000},
  "lastUpload": "2026-03-15T10:00:00Z"
}
```

GET `/api/advance/employee/{employee_code}?month=3&year=2026`

```
Response: {"advances": [...], "total": 5000, "count": 2}
```

DELETE `/api/advance/clear`

```
Response: {"success": true, "deleted": 7}
```

`DELETE /api/advance/{uid}`

```
Response: {"success": true, "deleted": 1}
```

---

9. FRONTEND PAGES & COMPONENTS

Routing (App.js)

Path	Page	Auth Required
`/login`	Login.jsx	No
`/`	Dashboard.jsx	Yes
`/employees`	EmployeeMaster.jsx	Yes
`/attendance`	AttendanceUpload.jsx	Yes
`/configuration`	SalaryConfiguration.jsx	Yes
`/reports`	SalaryReport.jsx	Yes
`/advance`	AdvanceManagement.jsx	Yes

Key Components

Component	File	Role
AuthProvider	context/AuthContext.jsx	Wraps app, provides `login`, `logout`, `authFetch`, `isAuthenticated`
AppProvider	context/AppContext.js	Global state: employees, config, attendance, results. Loads employees from server on mount.
Layout	components/Layout.jsx	Sidebar navigation (collapsible), theme toggle, language toggle (EN/HI), logout button
PrivateRoute	components/PrivateRoute.jsx	Redirects to `/login` if `!isAuthenticated`

State Management

- **Auth state:** `AuthContext` (token in `sessionStorage`)
- **App state:** `AppContext` (all in React state, NOT localStorage)
- **Only in localStorage:** `agm_theme` (light/dark), `agm_salary_config` (non-sensitive settings)
- **Sensitive data NEVER in localStorage:** employees, attendance, calculation results

---

10. SALARY CALCULATION ENGINE

File: `frontend/src/utils/salaryCalculator.js`

Core Formula

```
Per Day Salary    = Monthly Salary / Days in Month
Present Days      = Days in Month - Absent Days - Sandwich Days
Net OT Hours      = OT Hours - Short Hours
OT Days           = Net OT Hours / 9
Total Payable     = Present Days + Sunday Working + Holiday Working
+ OT Days
Total Salary      = Per Day Salary * Total Payable Days
Net Salary        = Total Salary - Advance Amount
```

Function Signature

```
calculateSalaries(attendanceData, employees, config, daysInMonth,
advancesMap = {})
```

Configuration Options (all toggleable)

| Setting | Default | Description | |---------|---------|-------------| | weekdayStandardHours | 9 | Full day = 9 hours on weekdays | | sundayStandardHours | 8 | Full day = 8 hours on Sundays | | shortHoursTolerance | 15 min | Within 15 min of standard = full day | | enableOvertime | true | Track OT hours beyond standard | | otConversionBase | 9 | Net OT hours / 9 = OT days | | enableSandwich | true | Sandwich rule for WO/HL between absences | | enableShortHoursDeduction | true | Track short hours for deduction | | weekdayMissingOutPunch | 'full' | Missing OUT = full day / half / absent | | sundayMissingOutPunch | 'full' | Same for Sundays | | zeroAttendanceZeroSalary | true | No attendance = zero salary | | onlySundayNoOT | per employee | If true, employee gets Sunday pay but no OT |

Day Classifications

| Type | Code | Description | |------|------|-------------| | Present | `PRESENT` | Employee came on a weekday | | Absent | `ABSENT` | No IN time on a working day | | Sunday Worked | `SUNDAY_WORKED` | Came on Sunday | | Holiday Worked | `HOLIDAY_WORKED` | Came on declared holiday | | Week Off | `WO` | Sunday, didn't come (paid) | | Holiday Off | `HL_OFF` | Declared holiday, didn't come (paid) |

Sandwich Rule

If a WO or HL day is surrounded by ABSENT days on both sides, it becomes unpaid (counted as absent).

Advance Deduction

Before calculation, `SalaryConfiguration.jsx` fetches advances from `GET /api/advance/list?month=X&year=Y` and builds a map:

```
advancesMap = { "101": 5000, "205": 3000 }  // employeeCode ->
total advance
```

Each employee result gets:

- `advanceAmount`: Total advance for this employee
- `netSalary`: `totalSalary - advanceAmount`

---

11. ADVANCE MANAGEMENT MODULE

Upload Flow

```
1. User uploads CSV/Excel on /advance page
2. Frontend sends FormData: file + employees_json (for matching
fallback)
3. Backend parses file with pandas
4. Filters rows where Type = "Salary" (case-insensitive)
5. Matches each row by Employee Code + Name (both must match)
6. Upserts by UID if provided, else checks for duplicates
7. Returns matched/updated/skipped/errors count
```

CSV Format

```
Date,Name,Advance,No,Type,UID
```

```
15/03/2026,Ramesh Kumar,5000,101,Salary,ADV001
16/03/2026,Suresh Singh,3000,102,Salary,ADV002
17/03/2026,Amit Sharma,2500,103,Stitching,ADV003  · SKIPPED (not
"Salary")
```

Column Matching (flexible)

The backend does fuzzy column name matching:

- `date` matches any column containing "date"
- `name` matches columns with "name" (not "sheet")
- `advance` matches "advance" or "amount"
- `code` matches "no", "no.", "emp", "code", "employee code"
- `type` matches columns with "type"
- `uid` matches "uid" or "id"

---

12. SECURITY MEASURES

12.1 Authentication

- JWT tokens (HS256) with 8-hour expiry
- Credentials from `.env` only (never hardcoded)
- Rate limiting on login: **5 requests/minute** per IP (HTTP 429 on exceed)
- Rate limiting on save endpoints: **10/minute**

12.2 CORS

```
allow_origins=["https://accounts.agmsale.com",
"https://agmone.agmsale.com"]
```

Only these two production domains are whitelisted.

12.3 Security Headers (every response)

| Header | Value | |--------|-------| | X-Frame-Options | SAMEORIGIN | | X-Content-Type-Options | nosniff | | Referrer-Policy | strict-origin-when-cross-origin | | Permissions-Policy | camera=(), microphone=(), geolocation=() | | Content-Security-Policy | default-src 'self'; script-src 'self' emergent + tailwind + posthog; connect-src 'self' posthog; font-src 'self' gstatic; style-src 'self' googleapis 'unsafe-inline' |

12.4 PostHog (Analytics)

- Session recording: **DISABLED** (`disable_session_recording: true`)
- Capturing: **OPTED OUT** (`posthog.opt_out_capturing()`)
- No employee/salary data is ever sent to PostHog

12.5 Client-Side Storage

| Storage Key | Location | Contains | |----------|----------|----------| | `auth_token` | sessionStorage | JWT token (cleared on tab close) | | `agm_theme` | localStorage | "dark" or "light" (non-sensitive) | | `agm_salary_config` | localStorage | Calculation config toggles (non-sensitive) | | ~~agm_employees~~ | **REMOVED** | Was in localStorage, now server-only | | ~~agm_attendance_data~~ | **REMOVED** | Was in localStorage, now React state

only | | ~~agm_last_calculation~~ | **REMOVED** | Was in localStorage, now React state only |

On app mount, old sensitive keys are explicitly cleaned up (`AppContext.js`).

---

13. DEPLOYMENT GUIDE

13.1 Prerequisites

- Python 3.11+
- Node.js 18+
- MongoDB 6.0+

13.2 Backend Setup

```
cd /app/backend
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
# Create .env file with production values
cp .env.example .env
# Edit .env: change APP_PASSWORD, JWT_SECRET_KEY, MONGO_URL
# Run
uvicorn server:app --host 0.0.0.0 --port 8001 --reload
```

13.3 Frontend Setup

```
cd /app/frontend
# Set backend URL
echo "REACT_APP_BACKEND_URL=https://your-api-domain.com" > .env
yarn install
yarn start       # Development (port 3000)
yarn build       # Production build (output in /build)
```

13.4 Production Checklist

- [ ] Change `APP_PASSWORD` to a strong password
- [ ] Change `JWT_SECRET_KEY` to a random 64-char string
- [ ] Set `MONGO_URL` to production MongoDB (with auth)
- [ ] Update `allow_origins` in `server.py` CORS if domains change
- [ ] Serve frontend build via Nginx/CDN
- [ ] Enable HTTPS (mandatory for secure cookies)
- [ ] Set up MongoDB indexes: `employees.code`, `salary_records.record_id`
- [ ] Set up log rotation for backend logs
- [ ] Configure supervisor or systemd for process management

13.5 Docker (example)

```
# Backend
FROM python:3.11-slim
WORKDIR /app
COPY backend/ .
```

```
RUN pip install -r requirements.txt
CMD ["uvicorn", "server:app", "--host", "0.0.0.0", "--port",
"8001"]
# Frontend
FROM node:18-alpine AS build
WORKDIR /app
COPY frontend/ .
RUN yarn install && yarn build
FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html
```

---

14. TESTING

14.1 Test Files

| File | Covers | |------|--------| | `tests/test_auth_security.py` | Login, token verification, protected endpoints, security headers | | `tests/test_rate_limit_localstorage.py` | Rate limiting on login/save, localStorage cleanup verification | | `tests/test_salary_history.py` | Salary save, history, compare, growth endpoints |

14.2 Running Tests

```
cd /app/backend
python -m pytest tests/ -v
```

14.3 Manual API Testing

```
# Get token
TOKEN=$(curl -s -X POST "http://localhost:8001/api/auth/login" \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"admin123"}' | python3 -c
"import sys,json;print(json.load(sys.stdin)['access_token'])")
# Use token
curl -s "http://localhost:8001/api/employees" \
  -H "Authorization: Bearer $TOKEN"
# Test rate limit (6th request should get 429)
for i in $(seq 1 6); do
  curl -s -o /dev/null -w "Attempt $i: %{http_code}\n" \
    -X POST "http://localhost:8001/api/auth/login" \
    -H "Content-Type: application/json" \
    -d '{"username":"x","password":"y"}'
done
```

14.4 Test Reports

Located at: `/app/test_reports/iteration_{n}.json`

- iteration_7: Auth + security headers (25/25 passed)
- iteration_8: Rate limiting + localStorage + advance deduction (all passed)

---

15. KNOWN CONSTRAINTS & DECISIONS

| Decision | Reason | |----------|--------| | Single admin user (from .env) | Business requirement: only one admin operates the system | | sessionStorage for JWT (not localStorage) | Clears on tab close = more secure for shared computers | | Attendance data in React state only | Security: no sensitive employee data persists in browser | | Employees loaded from MongoDB on each page load | Ensures data consistency, no stale localStorage cache | | Salary config in localStorage | Non-sensitive (just toggle values), persists across sessions for convenience | | `onlySundayNoOT` is per-employee | Some employees get Sunday pay but not OT (business rule) | | Advance matching requires BOTH name + code | Prevents wrong employee matches from CSV data | | Advance filter: only Type="Salary" | Other types (Stitching, etc.) are not salary advances | | Sandwich rule checks both sides of WO/HL | If absent on both sides, the off day becomes unpaid | | Excel attendance uses IN/OUT times only | Calculated work hours, not manually entered ones | | February fix: uses actual calendar days | Excel may have 31 columns but Feb has 28/29 |

End of Technical Skill File Document maintained by AGM Sales Engineering Team